Asymmetric Encryption Demo (RSA-2048)

Team Members

Eli Manning, Jacob Pugh, Brandon Magana

Overview

This program demonstrates public-key (asymmetric) cryptography using the RSA algorithm. It generates a 2048-bit RSA key pair, encrypts a user-supplied plaintext message with the public key, and decrypts the resulting ciphertext back to plaintext with the private key.

Extra-credit features include saving/loading keys to disk and creating and verifying digital signatures.

Dependencies

- **Java SE 8 or later** (no third-party libraries required)
- All cryptographic primitives come from the standard `java.security` and `javax.crypto` packages bundled with every JDK/JRE.

To verify your Java version:

```
java -version
```

How to Compile

Open a terminal in the directory containing `AsymmetricEncryption.java` and run:

```
javac AsymmetricEncryption.java
```

This produces `AsymmetricEncryption.class` in the same directory.

How to Run

```
java AsymmetricEncryption
```

The program presents an interactive, numbered menu. Use the number keys to select an action and press Enter.

Menu Options

Option	Action
1	Encrypt a message – enter plaintext, receive Base64 ciphertext
2	Decrypt a message – paste Base64 ciphertext, receive plaintext
3	Round-trip – encrypt then immediately decrypt, confirming correctness
4	Sign a message (Extra Credit)
5	Verify a signature (Extra Credit)
6	Run the three automated test cases
7	Regenerate the RSA key pair
8	Exit

Key Details

Parameter	Value
Key algorithm	RSA
Key size	2048 bits
Cipher transformation	RSA/ECB/OAEPWithSHA-256AndMGF1Padding
Signature algorithm	SHA256withRSA
Output encoding	Base64
Max plaintext size	190 bytes (UTF-8)

The maximum plaintext size for direct RSA encryption is dictated by the key size and padding overhead: 256 - 2x32 - 2 = 190 bytes.

Optional Features (Extra Credit)

Key Persistence

At startup the program checks for `public_key.der` and `private_key.der` in the current directory.

- If found, you are offered the option to **load** them instead of generating a fresh pair, allowing the same keys to be reused across program runs.
- After generating a new key pair you are asked whether to **save** them.
- Public key is stored in X.509/DER format; private key in PKCS8/DER format.

Digital Signatures

Options 4 and 5 implement **SHA-256 with RSA** digital signatures:

- **Sign (option 4):** The private key is used to sign a message. The resulting Base64-encoded signature can be shared alongside the message.
- **Verify (option 5):** The public key checks that a signature matches the original message. Any alteration to the message or signature will cause verification to fail.

Error Handling

Scenario	Behavior
Empty plaintext	Error: "Plaintext message cannot be empty."
Plaintext > 190 bytes	Error: explains the byte limit
Invalid Base64 on decrypt	Error: "Ciphertext is not valid Base64."
Wrong key or corrupted data	Descriptive error message; program continues
Empty message to sign	Error: "Message to sign cannot be empty."

Test Cases

Run option **6** from the menu to execute three automated tests:

1. **Short greeting** - `Hello, World!`

2. **Medium sentence** – `RSA encryption is a cornerstone of modern cybersecurity.`
3. **Special characters** – `Special chars: !@#$%^&*()_+ and numbers 1234567890.`

Each test encrypts the message, decrypts the result, and confirms the output matches the original. See `TestResults.txt` for sample output.

File List

File	Description
`AsymmetricEncryption.java`	Main source file
`TestResults.txt`	Sample test-run output
`README.md`	This file
`public_key.der`	Saved public key (created on first save)
`private_key.der`	Saved private key (created on first save)